

Video Game AI-Decision Tree CRUD For NPC Behaviour

Jahnavi
dept. School of SCDS
Sai University
Chennai, India
jahnavi.p-29@scds.saiuniversity.edu.in

Priyanka
dept. School of SCDS
Sai University
Chennai, India
chowdithapriyanka.s-29@scds.saiuniversity.edu.in

Abstract—This project presents the design and implementation of an intelligent Video Game Artificial Intelligence (AI) system using a Decision Tree model to simulate the behavior of Non-Player Characters (NPCs). The system is implemented in the C programming language and utilizes a Binary Tree data structure to represent decision-making logic in a structured and hierarchical manner. Each node in the tree corresponds to a specific action or decision, such as attacking an enemy, patrolling an area, or escaping from danger. The decision tree enables NP-Cs to evaluate conditions and choose appropriate actions dynamically, thereby enhancing realism and interactivity in gameplay. The system incorporates CRUD (Create, Read, Update, Delete) operations to allow flexible management of decision nodes. Dynamic memory allocation using functions such as free() ensures efficient memory usage and scalability. The project demonstrates how classical data structures can be effectively applied to AI systems in gaming, providing a foundation for more advanced AI techniques in the future.

Keywords—Decision Tree, NPC Behaviour, Game AI, Binary Tree, C Programming, CRUD Operations

I. INTRODUCTION

Artificial Intelligence has become a fundamental component in modern video games, significantly improving the realism and engagement of gameplay. NP-Cs, which are controlled by the game rather than the player, rely on AI to simulate intelligent behavior. In earlier games, NPC actions were predefined using static scripts, which resulted in predictable and repetitive interactions. This limitation reduced the overall gaming experience. To overcome these challenges, AI techniques such as decision trees are widely used. A decision tree provides a structured approach to decision-making, where each node represents a condition or action, and branches represent possible outcomes. This hierarchical representation allows NP-Cs to evaluate multiple conditions and choose appropriate actions dynamically. In this project, a binary tree is used to implement the decision tree, providing an efficient and organized structure for managing NPC behavior.

II. PROBLEM STATEMENT

In traditional game development, NPC behavior is often implemented using hard-coded logic. While this approach is simple, it lacks flexibility and scalability. Any modification to NPC behavior requires rewriting or restructuring the code, which increases development time and complexity. Additionally, hard-coded systems do not adapt well to dynamic game environments, resulting in unrealistic and repetitive NPC actions. This project aims to address these limitations by introducing a dynamic decision tree system. The proposed system allows developers to modify NPC behavior easily by adding, updating, or removing decision nodes without altering the entire program. This improves maintainability, scalability, and adaptability, making it suitable for complex and evolving game environments.

III. OBJECTIVE

The primary objective of this project is to design and implement a decision tree-based AI system for NPC behavior using a binary tree structure in C. The system aims to provide dynamic and flexible decision-making capabilities that can adapt to different game scenarios. Another objective is to implement efficient CRUD operations on the tree nodes, enabling easy modification and management of decisions. The project also focuses on utilizing dynamic memory allocation techniques to optimize memory usage and ensure efficient handling of data structures. Additionally, the system aims to demonstrate the practical application of data structures in real-world problems, particularly in game development and artificial intelligence.

IV. DATA STRUCTURE USED

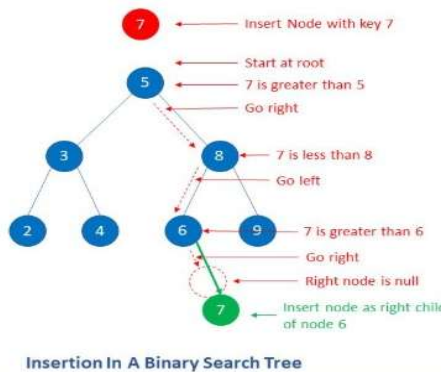
The system is built using a Binary Tree data structure, which is ideal for representing hierarchical relationships. In a binary tree, each node has at most two children, known as the left and right child. This structure is used to represent decisions and their possible outcomes in a clear and organized manner. Each node in the tree contains a unique identifier (ID), a decision or action description, and pointers to its left and right child nodes. The left child typically represents one possible outcome, while the right child represents another.

V.SYSTEM DESIGN

The system follows a tree-based design where the decision-making process is represented as a hierarchical structure. The root node represents the initial decision point, and child nodes represent subsequent actions or outcomes based on different conditions. This hierarchical design allows complex decision-making processes to be broken down into smaller, manageable components. The system is implemented as a menu-driven application, providing an interactive interface for users to perform various operations on the decision tree. Users can add new nodes, update existing decisions, delete nodes, search for specific nodes, and display the entire tree structure. The modular design ensures that each functionality is implemented separately, improving maintainability and readability.

VI.ALGORITHM

The system uses several algorithms to manage and manipulate the decision tree. The insertion algorithm adds a new node to the tree while maintaining its structure. The deletion algorithm removes a node and reorganizes the tree to preserve its properties. The update algorithm modifies the data stored in a node without affecting the tree structure. The search algorithm traverses the tree to locate a specific node based on given criteria. Traversal algorithms such as inorder, preorder, and postorder are used to display the tree structure. These algorithms ensure efficient navigation through the tree and enable users to visualize the decision-making process. The combination of these algorithms ensures that the system operates efficiently and effectively.



VI.IMPLEMENTATION

The implementation is carried out using the C programming language due to its efficiency and control over memory management. The binary tree is implemented using structures (`struct`) that define the properties of each node. Dynamic memory allocation is used to create and delete nodes during runtime, ensuring flexibility and efficient resource utilization. The program is divided into multiple functions, each responsible for a specific operation such as insertion, deletion, searching, and traversal. This modular approach enhances code readability and maintainability. Proper error handling is also included to manage invalid inputs and ensure the robustness of the system.



VIII. RESULTS

The system successfully demonstrates the implementation of a decision tree for NPC behavior. All CRUD operations are performed efficiently, and the tree structure is maintained correctly after each operation. The traversal functions display the tree in different formats, allowing users to understand the decision hierarchy clearly. The use of dynamic memory allocation ensures efficient utilization of system resources. The results indicate that the system is capable of handling dynamic updates and modifications without affecting performance. Overall, the project achieves its objectives and demonstrates the effectiveness of using binary trees for AI decision-making.

IX.CONCLUSION

This project highlights the importance of data structures in developing intelligent systems. By implementing a decision tree using a binary tree structure, the project demonstrates how NPC behavior can be modeled effectively in video games. The system provides flexibility, scalability, and efficiency, making it suitable for real-world applications. The successful implementation of CRUD operations and dynamic memory management further enhances the practicality of the system. This project serves as a foundation for developing more advanced AI systems and encourages the use of structured approaches in problem-solving.

X. FUTURE SCOPE

The project can be extended in several ways to improve its functionality and applicability. One major enhancement is the integration of graphical visualization tools to represent the decision tree visually. This would make it easier to understand and debug the system. Another improvement is the integration of the system with modern game engines such as Unity or Unreal Engine, allowing it to be used in real-world game development. Advanced AI techniques such as machine learning and reinforcement learning can also be incorporated to enable NPCs to learn from player interactions and improve their behavior over time. Additionally, optimizing the tree structure and implementing more complex decision-making models can further enhance the system's performance and capabilities.